

PROGRAM:

```
clc;

clear;

close all;

rng(1); % Fix random seed

% Parameters

N = 64; % Antennas

M = 4; % RF Chains

K = 4; % Users

SNR = 10; % dB

% Channel Matrix (Rayleigh)

H = (randn(N,K) + 1i*randn(N,K)) / sqrt(2);

% Analog Precoder

A = exp(1i*2*pi*rand(N,M)); % Phase-only (realistic RF)

% Digital Precoder (ZF)

W = (H' * H + 0.1*eye(K)) \ H';

% Hybrid Precoder

F = A * W;

% Normalize

F = F / norm(F,'fro');

% 3D BEAM RESPONSE PATTERN

theta = linspace(0,pi,100); % Elevation

phi = linspace(0,2*pi,100); % Azimuth

[TH,PH] = meshgrid(theta,phi);
```

```

% Uniform Linear Array Steering Vector

d = 0.5; % antenna spacing

AF = zeros(size(TH));

for n = 1:N
    AF = AF + F(n,1) * exp(1i*2*pi*d*(n-1)*sin(TH).*cos(PH));
end

AF_mag = abs(AF);

AF_mag = AF_mag ./ max(AF_mag(:));    % Normalize

% Convert to Cartesian

X = AF_mag .* sin(TH) .* cos(PH);
Y = AF_mag .* sin(TH) .* sin(PH);
Z = AF_mag .* cos(TH);

% Plot

figure('Color','w');
surf(X,Y,Z,AF_mag,'EdgeColor','none');
colormap jet;
colorbar;
caxis([0 1]);

title('3D Response Pattern','FontWeight','bold');

xlabel('X'); ylabel('Y'); zlabel('Z');

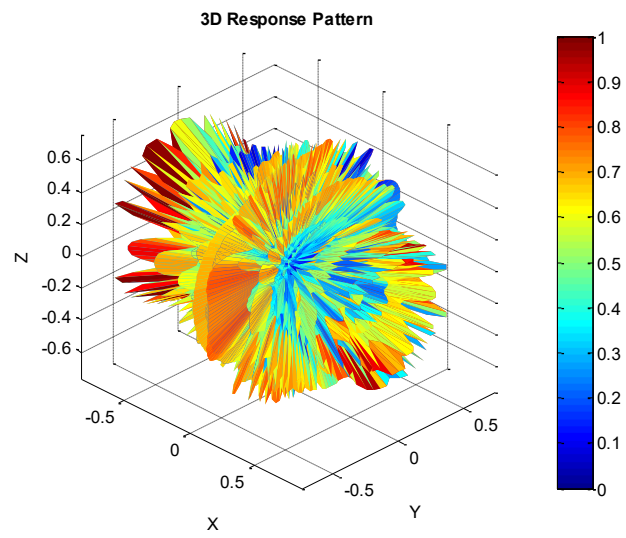
axis equal;

view(45,30);

grid on;

```

OUTPUT:



PROGRAM:

clc;

clear;

% Common Parameters

K = 10; % Number of users

N = 1e4; % Monte Carlo iterations

B = 20e6; % Bandwidth (Hz)

N0 = 1e-17; % Noise PSD

Pn = N0 * B; % Noise power

% SINR vs Number of Antennas (Full Monte Carlo)

antenna_range = 20:20:300;

Pt = 10; % Fixed transmit power (W)

avg_SINR_vs_Nt = zeros(size(antenna_range));

for a = 1:length(antenna_range)

 Nt = antenna_range(a);

 SINR_all = zeros(K, N);

 for iter = 1:N

 % Rayleigh channel

 H = (randn(K, Nt) + 1i*randn(K, Nt)) / sqrt(2);

 % Zero Forcing Precoder

 W_zf = H' * inv(H * H');

 % Power normalization

 W_zf = W_zf / norm(W_zf,'fro') * sqrt(Pt);

 % Effective channel

 Heff = H * W_zf;

 signal_power = abs(diag(Heff)).^2;

```

interference_power = sum(abs(Heff).^2,2) - signal_power;

SINR = signal_power ./ (interference_power + Pn);

SINR_all(:,iter) = SINR;

end

avg_SINR_vs_Nt(a) = mean(SINR_all(:));

end

figure;

plot(antenna_range,10*log10(avg_SINR_vs_Nt),'LineWidth',2);

grid on;

xlabel('Number of Transmit Antennas (N_t)');

ylabel('Average SINR (dB)');

title('SINR vs Number of Antennas (ZF Massive MIMO)');

% Throughput vs Transmit Power (Full Monte Carlo)

Nt = 200; % Fixed antenna number

Pt_dB_range = 0:2:30; % Power range in dB

avg_throughput_vs_Pt = zeros(size(Pt_dB_range));

for p = 1:length(Pt_dB_range)

    Pt = 10^(Pt_dB_range(p)/10);

    throughput_all = zeros(K, N);

    for iter = 1:N

        % Rayleigh channel

        H = (randn(K, Nt) + 1i*randn(K, Nt)) / sqrt(2);

        % ZF Precoder

        W_zf = H' * inv(H * H');

        W_zf = W_zf / norm(W_zf,'fro') * sqrt(Pt);

```

```

% Effective channel

Heff = H * W_zf;

signal_power = abs(diag(Heff)).^2;

interference_power = sum(abs(Heff).^2,2) - signal_power;

SINR = signal_power ./ (interference_power + Pn);

throughput = B * log2(1 + SINR);

throughput_all(:,iter) = throughput;

end

avg_throughput_vs_Pt(p) = mean(throughput_all(:));

end

figure;

plot(Pt_dB_range, avg_throughput_vs_Pt/1e6,'LineWidth',2);

grid on;

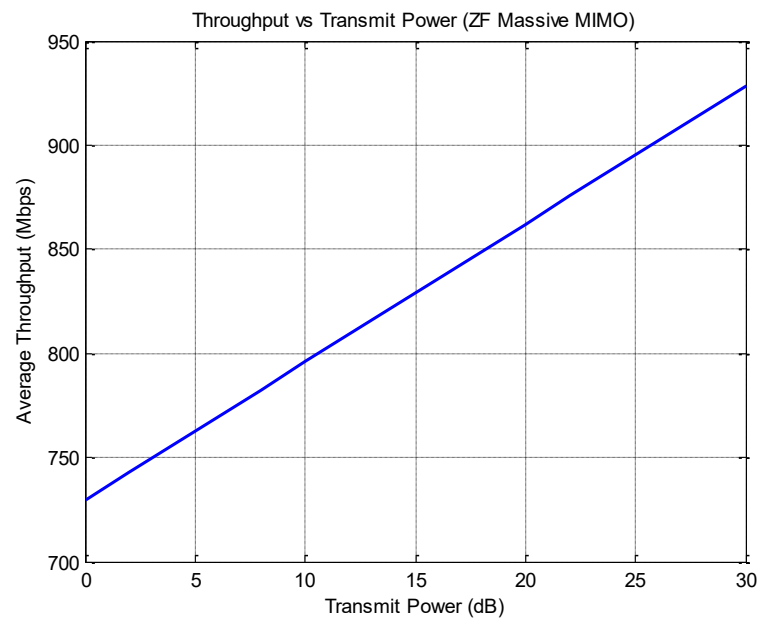
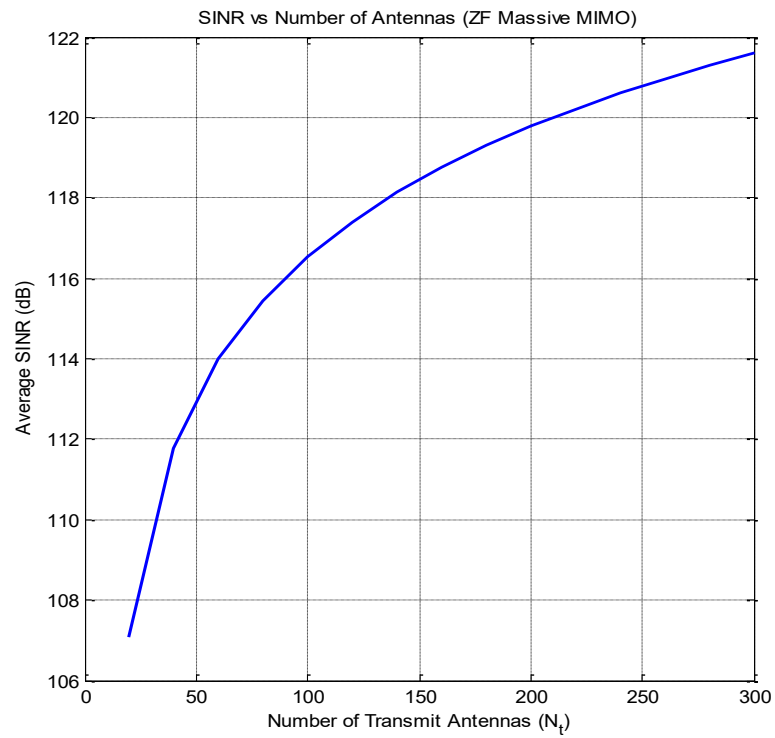
xlabel('Transmit Power (dB)');

ylabel('Average Throughput (Mbps)');

title('Throughput vs Transmit Power (ZF Massive MIMO)');

```

OUTPUT:



PROGRAM:

```
clc;

clear all;

close all;

% System Parameters

num_cells = 3;

num_UEs_per_cell = 10;

num_antenna_BS = 64;

num_realizations = 100;

% SNR range for plotting

snr_dB = 0:5:30;

snr = 10.^(snr_dB/10);

num_snr = length(snr);

% Cell parameters

cell_radius = 500;

cell_distance = 2*cell_radius;

% Store results

avg_rates = zeros(num_snr, num_cells);

for s = 1:num_snr

    total_rates = zeros(num_realizations, num_cells);

    for realization = 1:num_realizations

        % Generate UE locations

        UE_positions = cell(num_cells,1);

        for cell_idx = 1:num_cells

            UE_positions{cell_idx} = cell_radius * ...

                (rand(num_UEs_per_cell,1) + 1i*rand(num_UEs_per_cell,1));
```

```

end

% Channel generation (Rayleigh)
H = cell(num_cells,1);

for cell_idx = 1:num_cells

    H{cell_idx} = (randn(num_UEs_per_cell,num_antenna_BS) + ...
        1i*randn(num_UEs_per_cell,num_antenna_BS))/sqrt(2);

end

rates = zeros(num_cells,1);

for cell_idx = 1:num_cells

    H_current = H{cell_idx};

    % Zero Forcing Precoder

    W = pinv(H_current);

    % Normalize power

    W = W ./ norm(W,'fro');

    % Transmitted symbols

    s_vec = sqrt(1/num_UEs_per_cell)*ones(num_UEs_per_cell,1);

    % Received signal

    y = H_current * W * s_vec;

    % Noise

    noise = (randn(num_UEs_per_cell,1) + ...
        1i*randn(num_UEs_per_cell,1)) * sqrt(1/snr(s));

    received_signal = y + noise;

    % SINR calculation

    signal_power = abs(y).^2;

    noise_power = abs(noise).^2;

    sinr = signal_power ./ noise_power;

```

```

    % Achievable rate
    rate = log2(1 + sinr);
    rates(cell_idx) = mean(rate);
end

total_rates(realization,:) = rates;
end

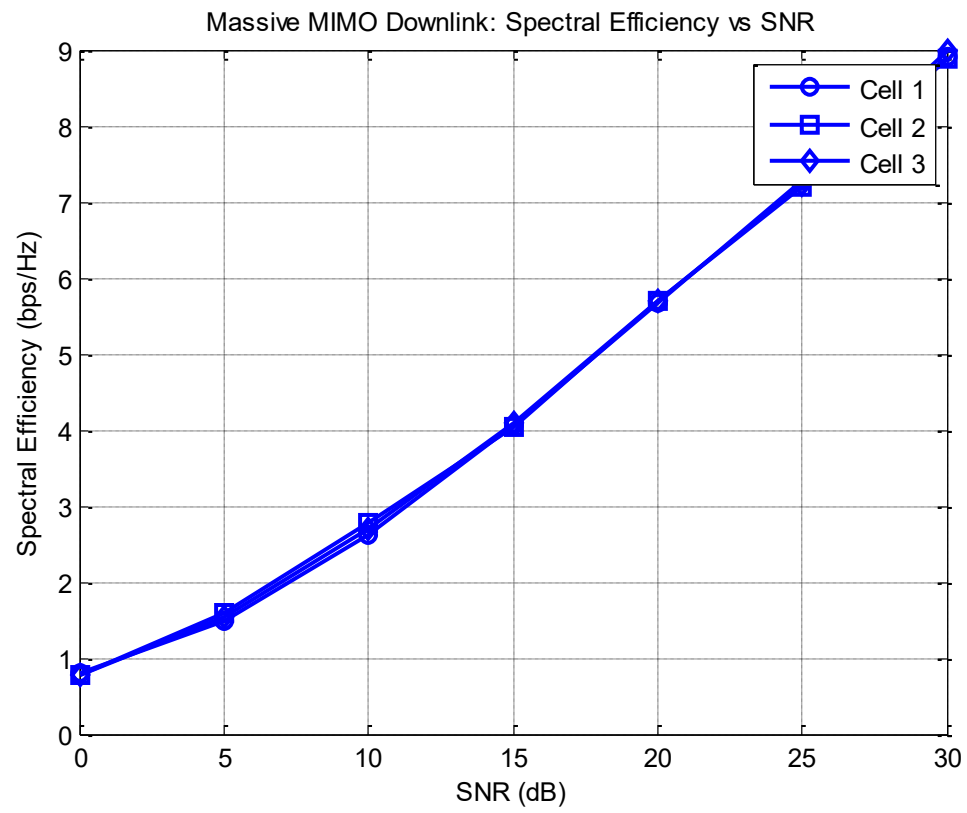
avg_rates(s,:) = mean(total_rates,1);
end

% Display results
disp('Average achievable rates (bps/Hz) per cell:')
disp(avg_rates)

% Plot graph
figure
plot(snr_dB, avg_rates(:,1),'-o','LineWidth',2)
hold on
plot(snr_dB, avg_rates(:,2),'-s','LineWidth',2)
plot(snr_dB, avg_rates(:,3),'-d','LineWidth',2)
grid on
xlabel('SNR (dB)')
ylabel('Spectral Efficiency (bps/Hz)')
title('Massive MIMO Downlink: Spectral Efficiency vs SNR')
legend('Cell 1','Cell 2','Cell 3')

```

OUTPUT:



PROGRAM:

```
clc;

clear;

close all;

% Parameters

num_UEs = 10; % Number of UEs

num_antennas = 64; % BS antennas

snr_dB_range = 0:2:30; % SNR range for graph

avg_rate = zeros(length(snr_dB_range),1);

for idx = 1:length(snr_dB_range)

    snr_dB = snr_dB_range(idx);

    snr = 10^(snr_dB/10); % Linear scale

    % Channel Matrix (Rayleigh Fading)

    H = (randn(num_UEs, num_antennas) + ...

        1i*randn(num_UEs, num_antennas)) / sqrt(2);

    % Zero Forcing Precoding

    W_zf = H' * inv(H * H'); % More stable for Massive MIMO

    % Normalize precoder (total transmit power = 1)

    W_zf = W_zf / sqrt(trace(W_zf * W_zf'));

    % Transmit Signal

    s = ones(num_UEs,1); % Unit symbols

    % Received Signal

    y = sqrt(snr) * H * W_zf * s;

    % Add AWGN Noise

    noise = (randn(num_UEs,1) + 1i*randn(num_UEs,1)) / sqrt(2);

    received_signal = y + noise;
```

```

% SINR Calculation (per user)

signal_power = abs(sqrt(snr)).^2;      % Since ZF removes interference
noise_power = 1;                      % Unit variance noise

sinr = signal_power / noise_power;

%% Achievable Rate

rate = log2(1 + sinr);

avg_rate(idx) = mean(rate);

end

% Plot Spectral Efficiency vs SNR

figure;

plot(snr_dB_range, avg_rate, 'LineWidth',2);

grid on;

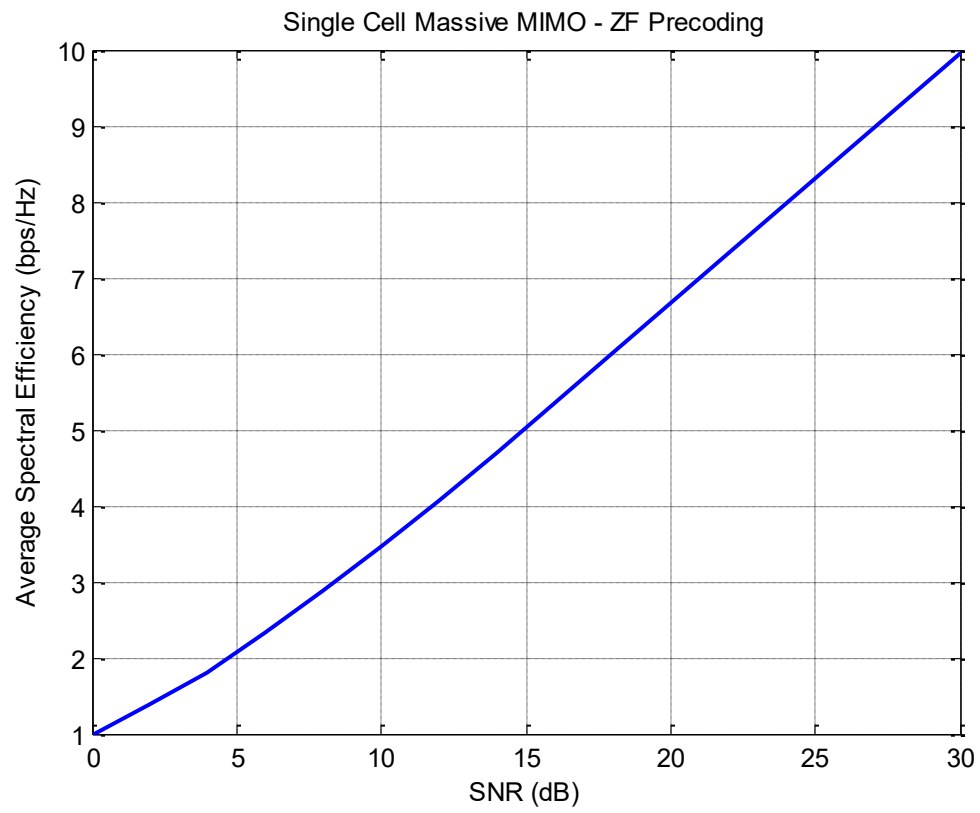
xlabel('SNR (dB)');

ylabel('Average Spectral Efficiency (bps/Hz)');

title('Single Cell Massive MIMO - ZF Precoding');

```

OUTPUT:



PROGRAM:

```
clc;

clear;

% Parameters

num_cells = 3;

num_UEs = 10;

num_antennas = 64;

snr_dB_range = 0:5:30;

num_realizations = 50;

avg_rates_vs_snr = zeros(length(snr_dB_range), num_cells);

for snr_idx = 1:length(snr_dB_range)

    snr_dB = snr_dB_range(snr_idx);

    snr = 10^(snr_dB/10);

    total_rates = zeros(num_realizations, num_cells);

    for realization = 1:num_realizations

        % Generate Rayleigh channels

        H = cell(num_cells,1);

        for c = 1:num_cells

            H{c} = (randn(num_UEs,num_antennas) + ...

                1i*randn(num_UEs,num_antennas))/sqrt(2);

        end

        rates = zeros(num_cells,1);

        for c = 1:num_cells

            Hc = H{c};

            % Zero-Forcing Precoder

            W = Hc' * inv(Hc*Hc');
```

```

% Column Normalization
for k = 1:num_UEs
    W(:,k) = W(:,k)/norm(W(:,k));
end

signal_power = zeros(num_UEs,1);
interference_power = zeros(num_UEs,1);
for k = 1:num_UEs
    hk = Hc(k,:);
    wk = W(:,k);

    % Desired Signal Power
    signal_power(k) = snr * abs(hk*wk)^2;

    % Inter-Cell Interference
    for c_int = 1:num_cells
        if c_int ~= c
            Hint = H{c_int};
            Wint = Hint' * inv(Hint*Hint');

            for m = 1:num_UEs
                wm = Wint(:,m);

                interference_power(k) = interference_power(k) + ...
                    snr * abs(hk*wm)^2;
            end
        end
    end
end

end

% SINR
sinr = signal_power ./ (interference_power + 1);

```

```

        % Shannon Capacity
        rate = log2(1 + sinr);
        rates(c) = mean(rate);
    end

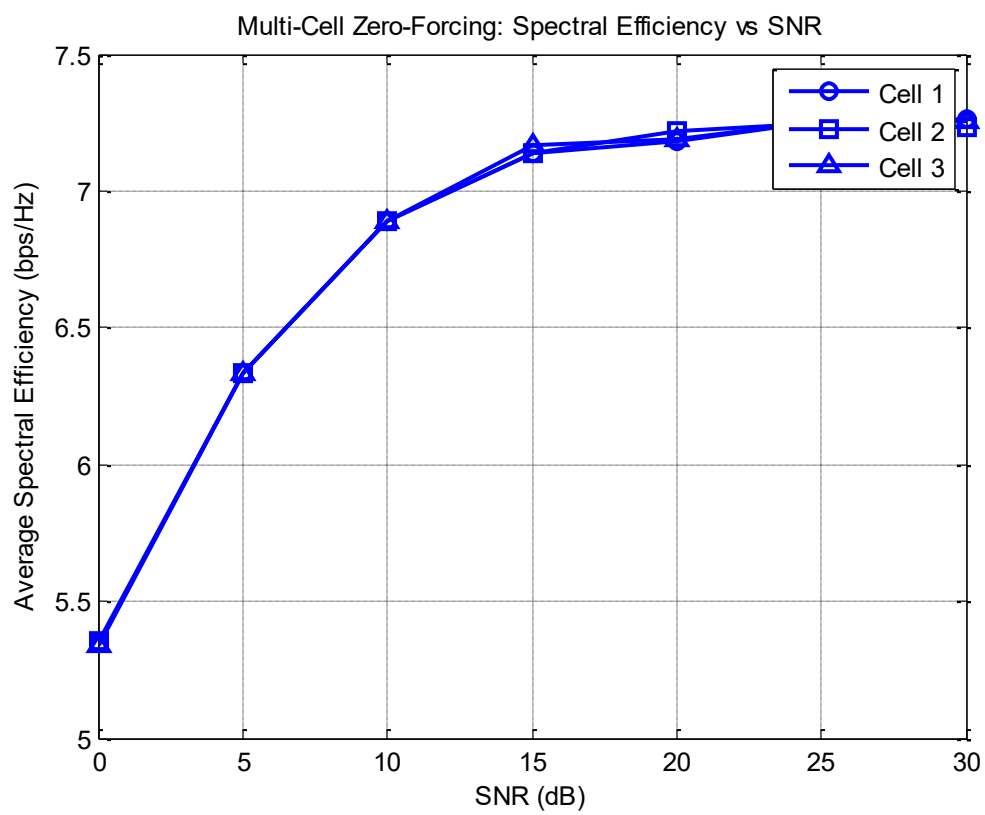
    total_rates(realization,:) = rates;
end

avg_rates_vs_snr(snr_idx,:) = mean(total_rates,1);
end

% GRAPH
figure;
plot(snr_dB_range, avg_rates_vs_snr(:,1),'-o','LineWidth',2);
hold on;
plot(snr_dB_range, avg_rates_vs_snr(:,2),'-s','LineWidth',2);
plot(snr_dB_range, avg_rates_vs_snr(:,3),'-^','LineWidth',2);
grid on;
xlabel('SNR (dB)');
ylabel('Average Spectral Efficiency (bps/Hz)');
title('Multi-Cell Zero-Forcing: Spectral Efficiency vs SNR');
legend('Cell 1','Cell 2','Cell 3');

```

OUTPUT:



PROGRAM:

```
clear all;

close all;

clc;

Rayleigh = 1;

AWGN = 0;

Nsc = 64;                                % Subcarriers

Ng = 16;                                % CP length

SNR_dB = 0:5:40;

Mt = 2;                                % Tx antennas

Mr = 2;                                % Rx antennas

pilots = 1:Nsc/Ng:Nsc;

DS = 5;                                % Delay spread

iteration_max = 200;

% Rayleigh Channel Model

if Rayleigh

    h = (randn(Mt*Mr,DS)+1i*randn(Mt*Mr,DS))/sqrt(2*DS);

end

MSE_LS = zeros(1,length(SNR_dB));

MSE_MMSE = zeros(1,length(SNR_dB));

for snr_idx = 1:length(SNR_dB)

    sigma2 = 10^(-SNR_dB(snr_idx)/10);

    err_LS = 0;

    err_MMSE = 0;
```

```

for iteration = 1:iteration_max

    H_act = zeros(Mt*Mr,Nsc);

    H_est_LS = zeros(Mt*Mr,Nsc);

    H_est_MMSE = zeros(Mt*Mr,Nsc);

    % Generate OFDM symbols

    data_time = zeros(Mt,Nsc+Ng);

    data_qam = zeros(Mt,Nsc);

    for tx = 1:Mt

        bits = randi([0 1],4,Nsc);

        % 16-QAM

        data_qam(tx,:) = ...

            (2*(bits(1,:)+2*bits(2,:))-3) + ...

            1i*(2*(bits(3,:)+2*bits(4,:))-3);

        % Insert pilots

        data_qam(tx,pilots) = 1+1i;

        temp = ifft(data_qam(tx,:));

        data_time(tx,:) = [temp(end-Ng+1:end) temp];

    end

    % Channel + Noise

    output = zeros(Mr,Nsc);

    for rx = 1:Mr

        for tx = 1:Mt

            ch = h((rx-1)*Mt+tx,:);

            conv_out = conv(data_time(tx,:),ch);

            output(rx,:) = output(rx,:) + conv_out(Ng+1:Ng+Nsc);

        end

    end

```

```

noise = sqrt(sigma2/2)*(randn(1,Nsc)+1i*randn(1,Nsc));

output(rx,:) = output(rx,:) + noise;

end

data_out = fft(output,[],2);

% Channel estimation

for tx = 1:Mt

    for rx = 1:Mr

        index = (rx-1)*Mt + tx;

        % Actual channel

        H_act(index,:) = fft([h(index,:) zeros(1,Nsc-DS)]);

        % LS estimation

        H_pilot = data_out(rx,pilots) ./ data_qam(tx,pilots);

        h_temp = ifft(H_pilot);

        h_temp = [h_temp zeros(1,Nsc-length(h_temp))];

        H_est_LS(index,:) = fft(h_temp);

        % MMSE refinement

        alpha = 0.9; % Smoothing factor

        H_est_MMSE(index,:) = alpha*H_est_LS(index,:);

        % Error accumulation

        err_LS = err_LS + mean(abs(H_act(index,:) - H_est_LS(index,:)).^2);

        err_MMSE = err_MMSE + mean(abs(H_act(index,:) - H_est_MMSE(index,:)).^2);

    end

end

end

MSE_LS(snr_idx) = err_LS/(Mt*Mr*iteration_max);

MSE_MMSE(snr_idx) = err_MMSE/(Mt*Mr*iteration_max);

```

```

end

% Plot 1: MSE vs SNR

figure;

plot(SNR_dB,10*log10(MSE_LS),'b-o','LineWidth',2);

hold on;

plot(SNR_dB,10*log10(MSE_MMSE),'r-s','LineWidth',2);

grid on;

xlabel('SNR (dB)');

ylabel('MSE (dB)');

legend('LS','MMSE');

title('MIMO-OFDM Channel Estimation Performance');

% Plot 2: Channel Magnitude Comparison

figure;

subplot(3,1,1);

plot(abs(H_act(1,:)));

title('Actual Channel Magnitude');

subplot(3,1,2);

plot(abs(H_est_LS(1,:)));

title('Estimated Channel (LS)');

subplot(3,1,3);

plot(abs(H_est_MMSE(1,:)));

title('Estimated Channel (MMSE)');

```

OUTPUT:

